TRAVAUX PRATIQUES — DÉVELOPPEMENT WEB

# Application CRUD de gestion des étudiants

HTML5 · CSS3 · JavaScript · PHP 8 (PDO) · MySQL

| Élément | Détail |
|---|---|
| Niveau | Débutant à intermédiaire (BTS 1 / BTS 2) |
| Durée indicative | 3 à 4 séances de 3 à 4 heures |
| Modalité | Individuel ou binôme |
| Matériel fourni | Projet de départ **tp-crud-etudiants.zip** (formulaire + maquette de la liste en HTML/CSS/JS, script SQL) |
| Enseignant | Ing. NGOUANG Barthélémy Fiacre |

## 1. Objectifs pédagogiques

À la fin de ce TP, l'étudiant sera capable de :

- structurer un formulaire HTML5 accessible (labels, fieldsets, types de champs adaptés) ;
- mettre en forme une interface responsive avec CSS (variables, Grid, Flexbox, media queries) ;
- valider des données côté client en JavaScript et expliquer pourquoi cela ne suffit pas ;
- se connecter à MySQL avec **PDO** et exécuter des **requêtes préparées** ;
- réaliser les quatre opérations **CRUD** : Create (ajouter), Read (lister/afficher), Update (modifier), Delete (supprimer) ;
- appliquer les règles de sécurité de base : requêtes préparées (injection SQL), htmlspecialchars (XSS), suppression en POST.

## 2. Prérequis et environnement

| Outil | Rôle | Remarque |
|---|---|---|
| XAMPP, WAMP ou Laragon | Serveur local Apache + PHP + MySQL | PHP 8.0 minimum |
| phpMyAdmin | Administrer la base de données | Inclus dans XAMPP/WAMP/Laragon |
| VS Code (ou autre) | Éditeur de code | Extension conseillée : PHP Intelephense |
| Navigateur récent | Tester l'application | Utiliser les outils de développement (F12) |

Connaissances attendues : bases du HTML/CSS, variables/conditions/boucles en PHP, notions de requêtes SQL (SELECT, INSERT, UPDATE, DELETE).

## 3. Contexte du projet

Un institut supérieur de Bafoussam souhaite remplacer ses fiches d'inscription papier par une petite application web. Le secrétariat doit pouvoir **enregistrer** un nouvel étudiant, **consulter** la liste des inscrits, **corriger** une fiche et **supprimer** une inscription annulée. Vous êtes chargé(e) de développer cette application.

## 4. Organisation du TP

| Partie | Contenu | Statut | Fichiers concernés |
|---|---|---|---|
| 1 | Étude du formulaire et de la liste fournis (HTML, CSS, JS) | **Fourni** — à étudier | index.html, liste.html, style.css, script.js |
| 2 | Base de données et connexion PDO | À réaliser | sql/base_etudiants.sql, connexion.php |
| 3 | CREATE — Enregistrer un étudiant | À réaliser | index.html → ajouter.php |
| 4 | READ — Lister, rechercher, afficher une fiche | À réaliser | liste.html → liste.php, voir.php |
| 5 | UPDATE — Modifier une fiche | À réaliser | modifier.php → mettre_a_jour.php |
| 6 | DELETE — Supprimer une fiche | À réaliser | supprimer.php |
| 7 | Bonus (au choix) | Facultatif | au choix |

> Un **guide pas à pas** accompagne ce TP : il détaille chaque étape avec le code et les tests à faire.

## 5. Structure du projet

### 5.1 Ce qui est fourni (Partie 1)

```
tp-crud-etudiants/
├── index.html              ← formulaire d'inscription
├── liste.html              ← maquette de la liste (données fictives)
├── assets/
│   ├── css/style.css       ← mise en forme (formulaire + liste)
│   └── js/script.js        ← validation côté client
├── sql/base_etudiants.sql  ← script de la base (Partie 2)
├── docs/                   ← cet énoncé
└── README.md
```

### 5.2 Structure attendue en fin de TP

```
tp-crud-etudiants/
├── assets/ ...             ← inchangé
├── sql/base_etudiants.sql  ← inchangé
├── connexion.php           ← connexion à la base (Partie 2)
├── index.html              ← formulaire branché sur ajouter.php (Partie 3)
├── ajouter.php             ← CREATE (Partie 3)
├── liste.php               ← READ : liste.html transformé (Partie 4)
├── voir.php                ← READ : fiche d'un étudiant (Partie 4)
├── modifier.php            ← UPDATE : formulaire prérempli (Partie 5)
├── mettre_a_jour.php       ← UPDATE : enregistrement (Partie 5)
└── supprimer.php           ← DELETE (Partie 6)
```

**Principe :** chaque page PHP est une page HTML du projet dans laquelle on ajoute un peu de PHP. HTML = structure, CSS = présentation, JavaScript = vérification dans le navigateur, PHP = traitement sur le serveur et accès à la base.

# Partie 1 — Étude du formulaire fourni

Le formulaire est entièrement fonctionnel côté client. Avant d'écrire la moindre ligne de PHP, vous devez **comprendre comment il est construit**, car votre code serveur dépendra directement des attributs `name` des champs.

## 1.1 Installation

1. Décompressez **tp-crud-etudiants.zip** dans le dossier web de votre serveur (`htdocs` pour XAMPP, `www` pour WAMP/Laragon).
2. Démarrez Apache, puis ouvrez `http://localhost/tp-crud-etudiants/`.
3. Cliquez sur **Enregistrer** sans rien remplir : tous les messages d'erreur doivent apparaître.
4. Remplissez correctement le formulaire et validez : une fenêtre affiche les données qui seront envoyées.
5. Cliquez sur « Liste des étudiants » dans le menu : la maquette **liste.html** s'affiche avec 4 étudiants fictifs et les boutons Voir / Modifier / Supprimer (ils ne font encore rien).

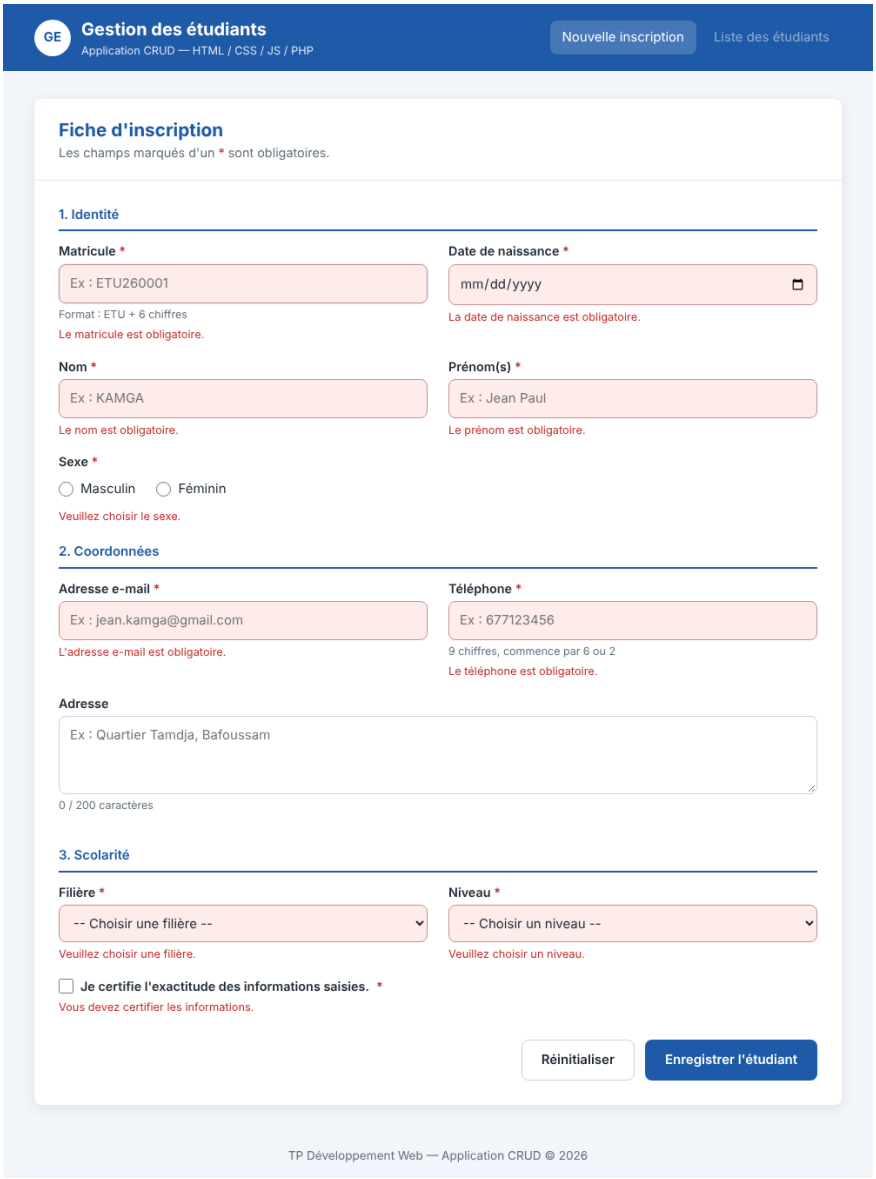



*Figure 1 — Formulaire soumis vide : la validation JavaScript signale chaque champ.*

## 1.2 Les champs du formulaire

Le tableau ci-dessous est votre **référence** pour toute la suite : les mêmes règles devront être réécrites en PHP.

| name | Type HTML | Oblig. | Règle de validation |
|---|---|---|---|
| matricule | text | Oui | ETU + 6 chiffres (ex : ETU260001), unique en base |
| nom | text | Oui | 2 à 50 caractères, lettres, espaces, apostrophe, tiret |
| prenom | text | Oui | 2 à 50 caractères, mêmes caractères que le nom |
| date_naissance | date | Oui | Format AAAA-MM-JJ ; âge entre 15 et 60 ans |
| sexe | radio | Oui | Valeur M ou F uniquement |
| email | email | Oui | Adresse valide, 100 caractères max, unique en base |
| telephone | tel | Oui | 9 chiffres commençant par 6 ou 2 |
| adresse | textarea | Non | 200 caractères maximum |
| filiere | select | Oui | GL, RT, IIA, CG ou MK |

| name | Type HTML | Oblig. | Règle de validation |
|---|---|---|---|
| niveau | select | Oui | BTS1, BTS2 ou LP3 |
| certification | checkbox | Oui | Doit être cochée (valeur 1) — non enregistrée en base |

### 1.3 Points clés du HTML (index.html)

```
<form id="form-etudiant" action="ajouter.php" method="POST"
      novalidate data-mode-demo="true">
    <div class="champ">
        <label for="nom">Nom <span class="obligatoire">*</span></label>
        <input type="text" id="nom" name="nom" maxlength="50">
        <small class="message-erreur" id="erreur-nom"></small>
    </div>
```

- **action="ajouter.php"** : fichier PHP qui recevra les données (créé en Partie 3).
- **method="POST"** : les données voyagent dans le corps de la requête, pas dans l'URL.
- **name="nom"** : c'est la clé qui sera lue en PHP avec `$_POST['nom']`. L'attribut `id` sert au label et au JavaScript.
- **label for="nom"** : lie le texte au champ (clic sur le label = focus du champ ; utile aux lecteurs d'écran).
- **novalidate** : désactive les bulles natives du navigateur pour laisser notre JavaScript afficher ses propres messages.
- **data-mode-demo="true"** : attribut personnalisé lu par le JS ; tant qu'il vaut *true*, l'envoi est intercepté.

### 1.4 Points clés du CSS (style.css)

- **Variables CSS** (`:root { --couleur-principale: … }`) : une couleur se change à un seul endroit.
- **Grid** : `.grille` crée 2 colonnes ; `.champ-large` occupe toute la largeur avec `grid-column: 1 / -1`.
- **États dynamiques** : les classes `.valide` et `.invalide` sont ajoutées par le JS pour colorer les champs.
- **Responsive** : sous 680 px, la media query passe la grille à 1 colonne et les boutons en pleine largeur.

### 1.5 Points clés du JavaScript (script.js)

```
const regles = {
    telephone(valeur) {
        if (valeur === '') return 'Le téléphone est obligatoire.';
        if (!/^[26]\d{8}$/.test(valeur)) return 'Numéro invalide : ...';
        return '';                        // chaîne vide = champ correct
    },
    // ... une fonction par champ
};
```

- Un objet **regles** associe à chaque `name` une fonction qui renvoie un message d'erreur ou une chaîne vide.
- **validerChamp()** applique la règle et met à jour le message et la classe CSS ; **validerFormulaire()** les appelle toutes.
- Écouteurs **blur**, **change** et **input** : validation en temps réel, sans gêner la saisie.
- À la soumission : si une erreur existe, `preventDefault()` bloque l'envoi et le curseur va sur le premier champ fautif.

- En mode démo, une fenêtre montre les paires `$_POST['clé'] → valeur` ; le contenu est inséré avec `textContent` (jamais `innerHTML` avec des données saisies).

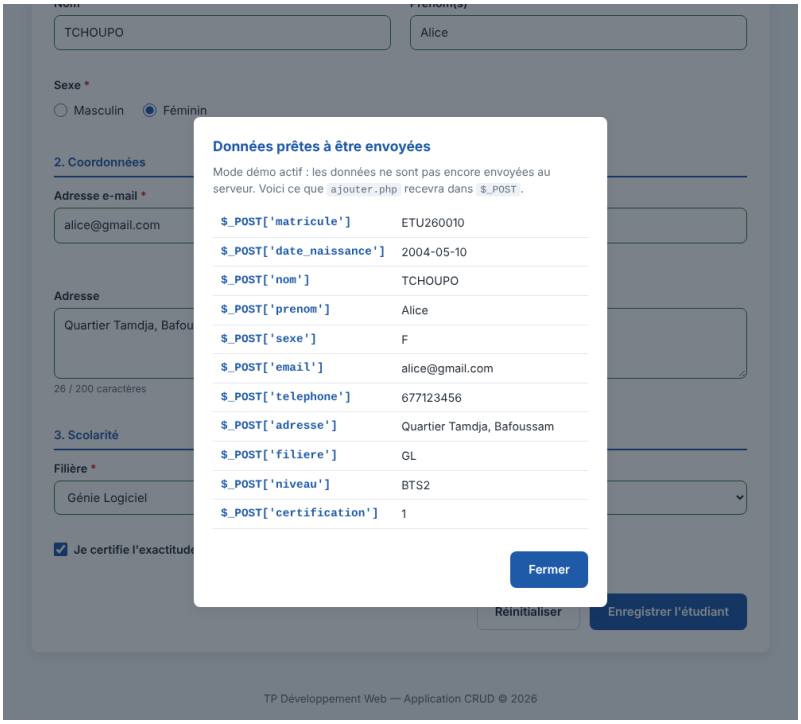



*Figure 2 — Mode démo : aperçu exact de ce que recevra $_POST.*

## 1.6 Travail demandé (questions à rendre)

1. Quelle est la différence entre les attributs `id` et `name` ? Lequel PHP utilise-t-il ?
2. Pourquoi le groupe « Sexe » a-t-il deux `input` avec le même `name` ? Que se passe-t-il si aucun n'est coché ?
3. Que contiendra `$_POST` si la case « certification » n'est pas cochée ?
4. Dans les outils de développement (F12), désactivez JavaScript puis soumettez un formulaire vide. Que constatez-vous ? Quelle conclusion en tirez-vous pour le PHP ?
5. Modifiez la variable `--couleur-principale` : combien d'éléments changent de couleur ? Pourquoi ?
6. Ouvrez **liste.html** : repérez la ligne `<tr>` « modèle » et les commentaires « En PHP ». Quels attributs des liens et du formulaire « Supprimer » devront changer ?

# Partie 2 — Base de données et connexion PDO

## 2.1 Créer la base

Ouvrez phpMyAdmin, onglet **SQL**, et exécutez le script `sql/base_etudiants.sql`. Extrait :

```
CREATE TABLE etudiants (
    id              INT UNSIGNED  NOT NULL AUTO_INCREMENT,
    matricule       CHAR(9)       NOT NULL,
    nom             VARCHAR(50)   NOT NULL,
    ...
    PRIMARY KEY (id),
    UNIQUE KEY uk_matricule (matricule),
    UNIQUE KEY uk_email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

**Questions :** (a) Pourquoi la colonne `id` est-elle la clé primaire et non le matricule ? (b) À quoi servent les deux contraintes `UNIQUE` ? (c) Quelle colonne de la table correspond à chaque attribut `name` du formulaire ?

## 2.2 Fichier connexion.php

Créez à la racine du projet le fichier `connexion.php`. Il sera inclus (`require`) au début de chaque page PHP qui utilise la base :

```
<?php
// connexion.php
$pdo = new PDO('mysql:host=localhost;dbname=tp_etudiants;charset=utf8mb4', 'root', '');
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
```

**Test :** créez un fichier temporaire `test.php` qui inclut la connexion, lit tous les étudiants avec `fetchAll()` et affiche leur nombre avec `count()`. Vous devez obtenir **4**. Supprimez ce fichier ensuite.

# Partie 3 — CREATE : enregistrer un étudiant

## 3.1 Brancher le formulaire

1. Dans `index.html`, passez l'attribut du formulaire à `data-mode-demo="false"` : les données partent maintenant vers `ajouter.php`.
2. Dans le menu, faites pointer « Liste des étudiants » vers `liste.php`.

## 3.2 Travail à réaliser dans ajouter.php

1. Inclure `connexion.php`.
2. Récupérer chaque champ dans une variable : `$nom = $_POST['nom'] ?? '';`
3. Vérifier que les champs obligatoires ne sont pas vides ; sinon, préparer un message d'erreur.
4. Vérifier dans la base que le matricule et l'e-mail ne sont pas déjà utilisés (SELECT préparé).
5. S'il n'y a pas d'erreur : exécuter l'INSERT avec une **requête préparée**, puis rediriger vers `liste.php?msg=ajout`.
6. Sinon : afficher le message d'erreur et un bouton pour revenir au formulaire.

```
$requete = $pdo->prepare('INSERT INTO etudiants
    (matricule, nom, prenom, date_naissance, sexe, email, telephone, adresse, filiere, niveau)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)');
$requete->execute([ /* À COMPLÉTER : les 10 variables, dans le même ordre */ ]);
```

**Requêtes préparées :** on écrit des ? à la place des valeurs et on les donne dans `execute()`. Ne mettez **jamais** une variable directement dans une requête SQL : c'est la porte ouverte à l'**injection SQL**.

**Pourquoi vérifier aussi en PHP ?** Le JavaScript s'exécute chez l'utilisateur : il peut être désactivé (question 4 de la Partie 1). Le serveur ne doit jamais faire confiance aux données reçues.

# Partie 4 — READ : lister, rechercher, afficher

## 4.1 liste.php

- Enregistrez **liste.html** sous le nom `liste.php`.
- En haut du fichier, lisez tous les étudiants triés par nom (`SELECT ... ORDER BY nom`) avec `fetchAll()`.
- Ne gardez qu'**une seule** ligne `<tr>` et placez-la dans une boucle `foreach` ; remplacez les textes fictifs par les valeurs de chaque étudiant.
- Affichez le nombre d'étudiants avec `count()`, et « Aucun étudiant trouvé. » si la liste est vide.
- Liens : `voir.php?id=…` et `modifier.php?id=…` avec l'id de l'étudiant.
- Recherche (formulaire en **GET**, champ `q`) : filtrer sur le matricule, le nom ou le prénom avec `LIKE`.
- Afficher le message vert seulement si l'adresse contient `?msg=ajout` (puis `modif`, `suppression`).

> **Sécurité XSS :** toute donnée venant de la base ou de l'utilisateur est affichée avec `htmlspecialchars()`. Exemple : `<?php echo htmlspecialchars($etudiant['nom']); ?>`

## 4.2 voir.php?id=…

- Lire l'id dans `$_GET['id']` et chercher l'étudiant avec `fetch()`.
- S'il n'existe pas, retourner à la liste.
- Afficher sa fiche complète (tableau `class="fiche"`, déjà prévu dans le CSS).

# Partie 5 — UPDATE : modifier une fiche

1. `modifier.php` : copie de `index.html`. Charger l'étudiant comme dans `voir.php`.
2. Remplir le formulaire : attribut `value` des champs, `checked` sur le bon bouton radio, `selected` sur les bonnes options.
3. Envoyer le formulaire à `mettre_a_jour.php` avec un champ caché `<input type="hidden" name="id">`.
4. `mettre_a_jour.php` : mêmes vérifications que `ajouter.php`, mais le doublon se cherche chez les **autres** étudiants (`AND id != ?`).
5. Exécuter `UPDATE etudiants SET ... WHERE id = ?`, puis rediriger vers `liste.php?msg=modif`.

> **Attention :** sans `WHERE id = ?`, un UPDATE modifie **toutes** les lignes de la table !

# Partie 6 — DELETE : supprimer une fiche

- Dans `liste.php`, le bouton « Supprimer » est un petit formulaire **POST** vers `supprimer.php` avec un champ caché `id` et une confirmation : `onsubmit="return confirm('Supprimer cet étudiant ?');"`
- `supprimer.php` lit l'id dans `$_POST`, exécute `DELETE FROM etudiants WHERE id = ?` et redirige vers `liste.php?msg=suppression`.

> **Pourquoi pas un simple lien** `supprimer.php?id=3` ? Un lien peut être ouvert par erreur ou par un robot. Toute action qui **modifie ou supprime** des données passe par POST.

## Partie 7 — Bonus (au choix)

- Vérifier aussi en PHP le **format** du matricule, de l'e-mail et du téléphone.
- Ajouter le champ **lieu_naissance** : formulaire, colonne dans la table (`ALTER TABLE`), INSERT, UPDATE et affichage.
- Afficher le libellé complet de la filière (« Génie Logiciel ») au lieu du code (« GL »).
- Mettre l'en-tête et le pied de page dans des fichiers séparés inclus avec `require`.
- **Pagination** de la liste (10 étudiants par page, `LIMIT` / `OFFSET`).

## Liste de contrôle

| Vérification | Où |
|---|---|
| Toutes les requêtes SQL utilisent prepare() + execute() avec des ? | Partout |
| Toutes les données affichées passent par htmlspecialchars() | liste, voir, modifier |
| Les champs obligatoires sont vérifiés en PHP, même si le JS les vérifie | ajouter, mettre_a_jour |
| Chaque UPDATE et chaque DELETE a un WHERE id = ? | mettre_a_jour, supprimer |
| La suppression passe par un formulaire POST avec confirmation | liste, supprimer |
| Après un enregistrement, on redirige vers la liste avec un message | ajouter, mettre_a_jour, supprimer |
| Le code est indenté et commenté | Partout |

## Livrables

- Le dossier complet du projet compressé : **NOM_Prenom_TP_CRUD.zip** ;
- un court compte rendu (PDF, 3 pages maximum) avec les réponses aux questions des Parties 1 et 2, et une capture d'écran de chaque page réalisée.

## Barème indicatif (/20)

| Partie | Critères | Points |
|---|---|---|
| 1 | Réponses aux questions de la Partie 1 | 2 |
| 2 | Base créée, connexion fonctionnelle, réponses aux questions 2.1 | 2 |
| 3 | CREATE : champs vérifiés, doublon détecté, INSERT préparé, redirection avec message | 4 |
| 4 | READ : liste depuis la base, recherche, fiche détaillée, htmlspecialchars | 5 |
| 5 | UPDATE : formulaire prérempli (radio et listes compris), UPDATE préparé | 3 |
| 6 | DELETE : formulaire POST + confirmation + DELETE préparé | 2 |
| — | Propreté du code (indentation, commentaires) et compte rendu | 2 |
| 7 | Bonus (jusqu'à +2, note plafonnée à 20) | +2 |

**Conseil final :** avancez partie par partie et testez chaque étape avant de passer à la suivante. Un CRUD simple qui fonctionne vaut mieux qu'un projet ambitieux inachevé.